A1.5 CONTRATO DE LA FUNCIÓN DE INGESTA

PROGRAMA EDUCATIVO

Ingeniería en Sistemas Computacionales

NOMBRE DEL PROYECTO

Automatización de mantenimiento preventivo con monitoreo permanente utilizando APIs y scripts

EMPRESA

Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.

PRESENTA

Axel Gabriel Ramírez Ortiz
Matrícula: 122043202 · Grupo: S206

ASESOR UPQ:

Ana Laura Lira Cortes

CO-ASESOR UPQ:

Ivan Peredo Valderrama

ASESORA EMPRESARIAL:

M.T.A. Sandra Lucía de la Fuente Bermúdez

Querétaro, Qro., 29 de octubre del 2026

ÍNDICE

1. OBJETIVO DE LA ACTIVIDAD

2. DESARROLLO DE LA ACTIVIDAD

- Los dos caminos de comunicación
- Reglas comunes
- Camino A: el reporte del equipo
- Evitar duplicados y reintentos
- Camino B: las funciones del agente para los comandos
- Funciones de la API
- Respuestas y análisis preliminar de riesgos

3. RESULTADOS DE LA ACTIVIDAD

4. PENDIENTES Y SIGUIENTES PASOS

REFERENCIAS

1. OBJETIVO DE LA ACTIVIDAD

En esta actividad definí cómo se comunica el agente con la plataforma: qué información manda, en qué formato, cómo demuestra que es quien dice ser y qué hace cuando se cae la conexión. A este acuerdo le llamo **contrato**, porque las dos partes tienen que respetarlo para poder entenderse; si el agente cambia algo por su cuenta, la plataforma deja de entenderlo.

Responde a la segunda pregunta del proyecto, ¿qué reglas debe seguir el envío de reportes para que cada resultado llegue completo y se pueda rastrear?, y prepara dos hipótesis: la H1, de que Windows y Mac pueden mandar el mismo formato, y la H2, de que se puede entregar al menos el 99 % de los reportes aunque la conexión falle por momentos.

Tabla 1. Datos de la actividad.

Concepto	Descripción
Clave de la actividad	A1.5
Fase del protocolo	F1. Requerimientos y arquitectura
Semanas	S3 – S4 (21 de septiembre al 2 de octubre de 2026)
Horas planeadas	20 horas
Producto	Especificación de la API
Entregable al que pertenece	E1. Documento de requerimientos y arquitectura (sub-entregable E1.5)

Fuente: Protocolo de estadía profesional, Bitlab-IT (2026).

2. DESARROLLO DE LA ACTIVIDAD

Los dos caminos de comunicación

Al revisar lo que necesitaba el módulo me di cuenta de que el agente no tiene una sola conversación con la plataforma, sino dos muy distintas. Por un lado tiene que mandar cada cierto tiempo cómo está el equipo; por otro, tiene que estar pendiente de los comandos que lance un técnico y devolver la respuesta en vivo. Por eso los separé en dos caminos, ambos por HTTPS.

Tabla 2. Caminos de comunicación entre el agente y la plataforma.

Camino	Para qué sirve	Cómo funciona	Estado
A. Reporte del equipo	Enviar cada 5 minutos el resultado de las verificaciones de la A1.3.	Una función de ingesta en Supabase recibe y revisa el reporte.	Diseñado; se programa en el E5.
B. Ejecución de comandos	Recibir comandos pendientes y devolver su salida en vivo.	Tres funciones de la base de datos.	Ya funciona en la demostración.

Fuente: Elaboración propia.

Reglas comunes

Los dos caminos siguen las mismas reglas básicas, que tomé de las buenas prácticas para sistemas que procesan datos de forma confiable (Kleppmann, 2017). Toda la información viaja por HTTPS, solo de salida y por el puerto 443. Los datos se escriben en JSON, que es fácil de leer tanto para una persona como para un programa (Bray, 2017), y las fechas siempre van en hora universal, por ejemplo 2026-10-28T18:05:00Z, para que no haya confusiones con los husos horarios de cada cliente.

Para identificar a cada equipo elegí una **clave propia por computadora**, de 48 caracteres, que se genera al registrarla. En la base de datos solo se guarda su huella y la clave nunca aparece en los registros ni en la salida de los comandos. Si alguna vez se sospecha que una clave se filtró, basta con volver a registrar el equipo para que se genere otra y la anterior deje de funcionar. Además, cada reporte dice qué versión del formato usa, para poder mejorar el contrato en el futuro sin romper a los agentes que ya estén instalados.

Camino A: el reporte del equipo

Cada 5 minutos el agente arma un reporte y lo manda a la función de ingesta, que es la puerta de entrada de los reportes. La Tabla 3 describe lo que lleva.

Tabla 3. Datos que lleva cada reporte.

Dato	Obligatorio	Descripción
id_reporte	Sí	Identificador único que crea el agente y que se conserva aunque el reporte se reenvíe. Es lo que evita los duplicados.
version_esquema	Sí	Versión del formato; en esta primera versión es 1.0.

Dato	Obligatorio	Descripción
generado_en y duracion_ms	Sí	Cuándo se terminó el reporte y cuánto tardó el agente en armarlo. Sirve para medir el impacto en el equipo (H5).
dispositivo	Sí	Nombre del equipo, sistema, versión del sistema, tipo de procesador y versión del agente.
metricas	Sí	Procesador, memoria, disco, tiempo encendido, procesos que más consumen y servicios importantes.
tareas	No	Resultado de las tareas de mantenimiento programadas que se hayan ejecutado.
errores_agente	No	Problemas que tuvo el propio agente al armar el reporte.

Fuente: Elaboración propia.

Este es un ejemplo de reporte de una computadora con Windows. Una Mac manda exactamente la misma estructura y solo cambian los valores, como el sistema o el nombre del disco; justo eso es lo que se va a comprobar con la hipótesis H1.


```
{
 "id_reporte": "5b0f3c1e-6a4d-4f2b-9a77-0c8f1d2e3a41",
 "version_esquema": "1.0",
 "generado_en": "2026-10-28T18:05:00Z",
 "duracion_ms": 2140,
 "dispositivo": { "nombre": "PC-Server", "sistema": "windows", "version_so": "Windows 11 24H2",
                  "arquitectura": "x64", "version_agente": "0.1.0" },
 "metricas": {
  "cpu_pct": 37.5,
  "memoria_total_mb": 16384, "memoria_usada_pct": 71.2,
  "discos": [ { "unidad": "C:", "total_gb": 476.0, "libre_gb": 251.3, "libre_pct": 52.8 } ],
  "tiempo_encendido_seg": 412800,
  "procesos_top": [ { "pid": 4120, "nombre": "chrome", "cpu_pct": 12.4, "memoria_mb": 1830, "responde":
true } ],
  "servicios": [ { "nombre": "Spooler", "estado": "en_ejecucion" } ]
 },
 "tareas": [],
 "errores_agente": []
}
```


La Figura 1 muestra, con un diagrama de secuencia, todo lo que pasa desde que el agente arma el reporte hasta que el panel se actualiza, incluyendo los casos en que algo sale mal: sin red, con una clave inválida o con un reporte repetido.

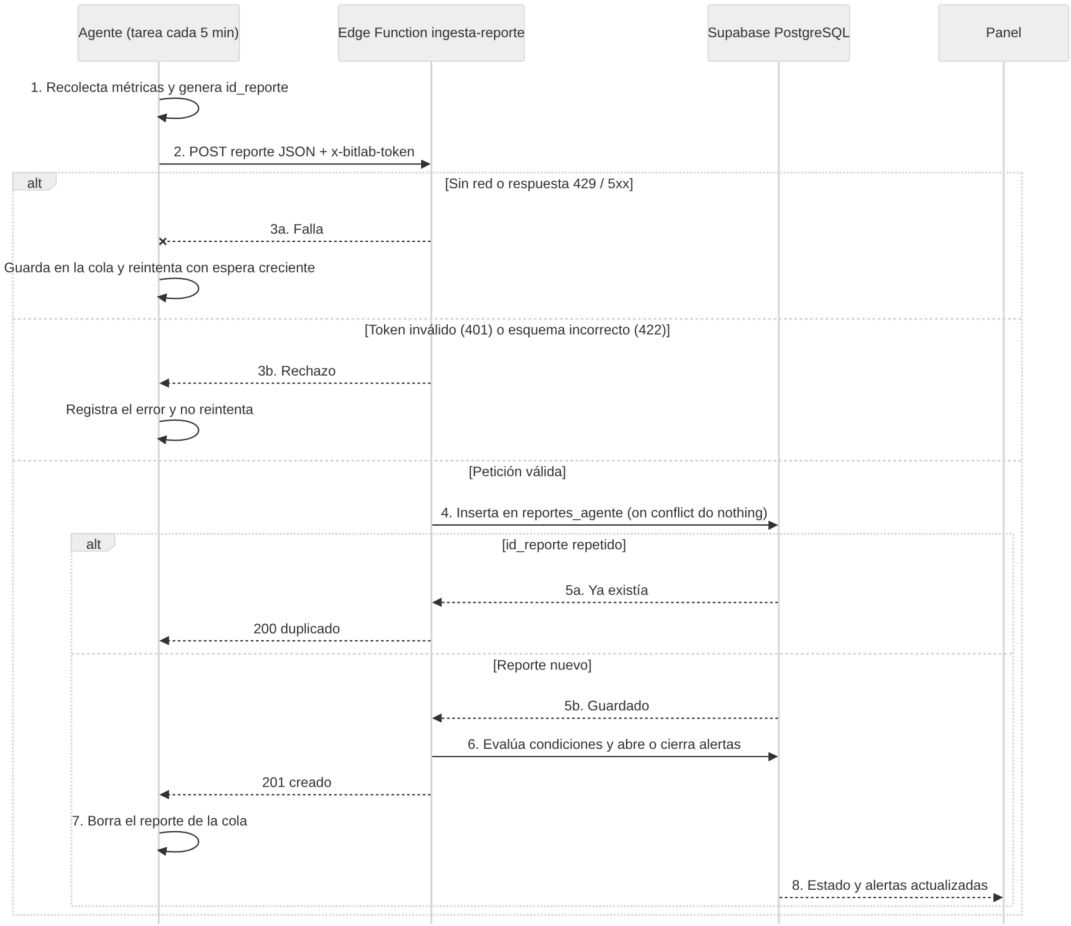


Figura 1. Diagrama de secuencia del envío de un reporte a la función de ingesta. *Fuente: Elaboración propia.*

Evitar duplicados y reintentos

Uno de los casos que más me preocupaba era este: el reporte llega a la plataforma, pero la conexión se corta antes de que el agente reciba la respuesta. El agente piensa que falló y lo vuelve a mandar. Para que el reporte quede guardado **una sola vez**, el agente crea el identificador del reporte una sola vez y lo reutiliza en cada intento, y la base de datos no acepta dos reportes con el mismo identificador. Si llega uno repetido, simplemente contesta que ya lo tenía. En el camino de los comandos apliqué la misma idea con el número de orden de cada línea, y eso ya está implementado.

Para los reintentos pensé en las clínicas con internet inestable. El agente solo vuelve a intentar cuando tiene sentido: si falló la red, se acabó el tiempo o la plataforma está saturada. Si el problema es la clave o el formato, reenviarlo no cambiaría nada. Entre cada intento espera 2, 4, 8 segundos y así hasta 60, más un pequeño tiempo al azar

para que muchos equipos no reintenten al mismo tiempo. Mientras no haya red, guarda hasta 288 reportes, que son 24 horas; si se llena, descarta el más antiguo y lo anota, para que la pérdida se pueda medir. Al volver la conexión, manda primero los más viejos.

Con estas reglas se va a medir la hipótesis H2 en el piloto: reportes guardados entre reportes generados, con una meta de **99 % o más**.

Camino B: las funciones del agente para los comandos

Este camino ya funciona en la demostración de la consola remota. La Figura 2 muestra el diagrama de secuencia de cuando un técnico lanza un comando: desde que lo elige en el panel hasta que ve el resultado en la consola, con los casos de un comando fuera del catálogo o una clave inválida.

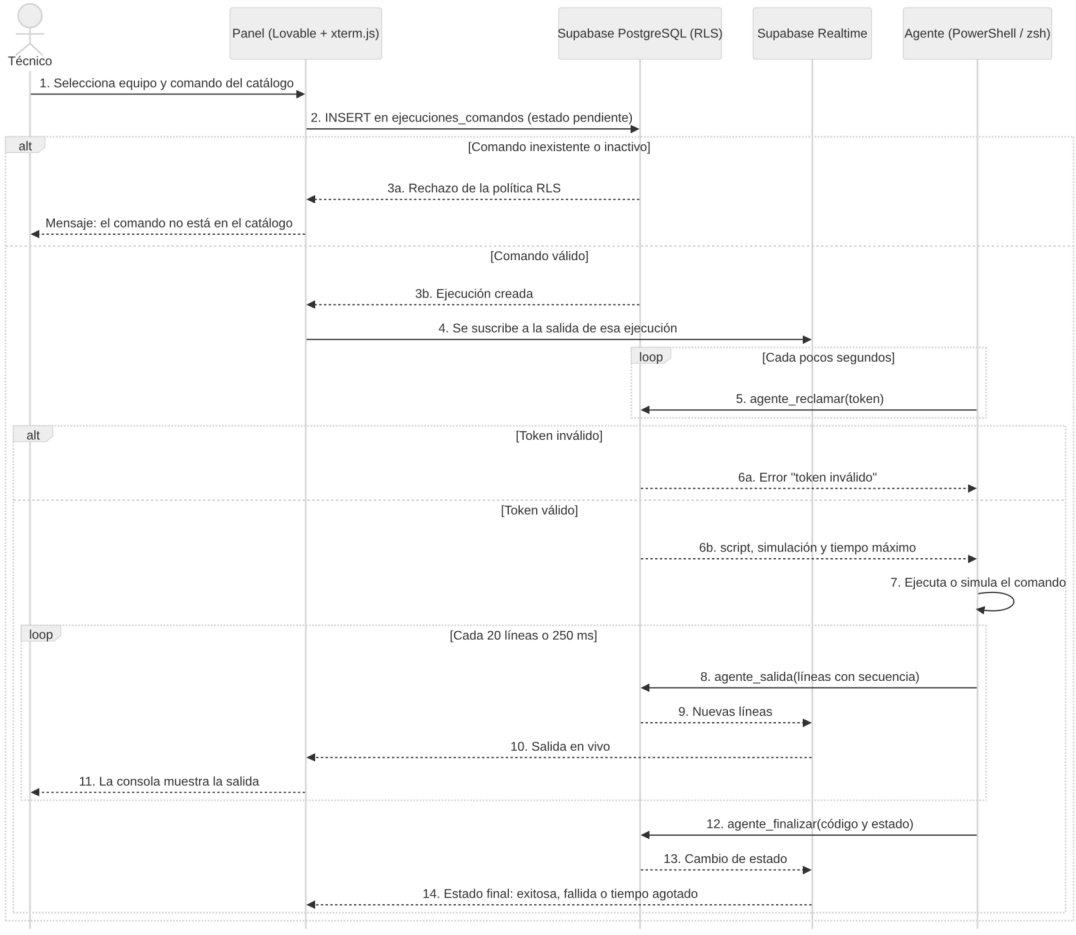


Figura 2. Diagrama de secuencia de la ejecución de un comando remoto. *Fuente: Elaboración propia.*

El agente de la demostración manda la salida en grupos de 20 líneas o cada cuarto de segundo, lo que pase primero, y reintenta cada envío hasta tres veces. Gracias a eso, el técnico ve la respuesta prácticamente al mismo tiempo que el equipo la genera. Hoy el agente pregunta cada 2 segundos; para producción propongo hacerlo cada 5, para no cargar de más al equipo ni a la base, y lo voy a medir en el piloto.

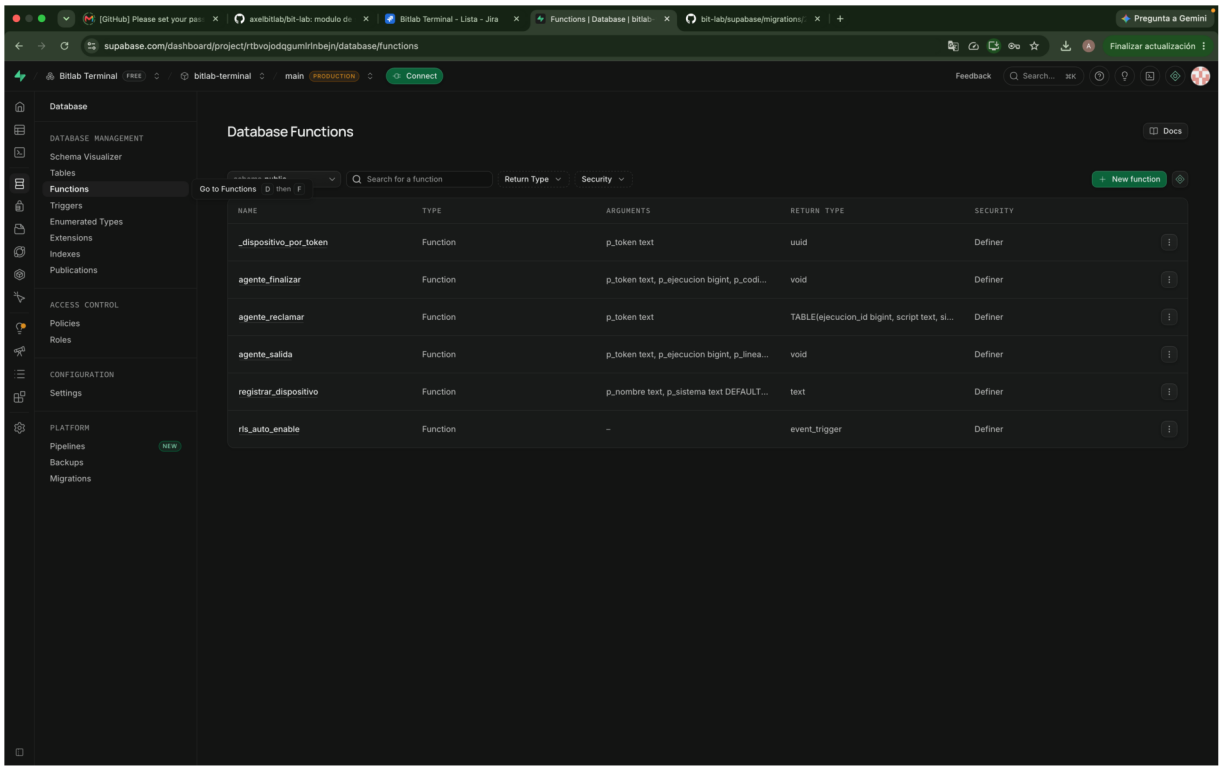


Figura 3. Funciones del agente creadas en Supabase. *Fuente: Elaboración propia.*

Funciones de la API

Para que quede claro qué recibe y qué devuelve cada punto de entrada, la Tabla 4 resume todas las funciones del contrato, tanto las del agente como las que usa el panel.

Tabla 4. Funciones de la API del módulo.

Método	Ruta	Descripción	Parámetros o cuerpo	Respuesta exitosa
POST	/functions/v1/ingesta-reporte	Recibe y revisa el reporte periódico del agente.	Encabezados: apikey y x-bitlab-token. Cuerpo: reporte JSON (Tabla 3).	201 Created → { id_reporte, alertas_abiertas }

Método	Ruta	Descripción	Parámetros o cuerpo	Respuesta exitosa
POST	/rest/v1/rpc/agente_reclamar	Entrega el siguiente comando pendiente del equipo y lo marca en ejecución.	Encabezado: apikey. Cuerpo: { p_token }	200 OK → [{ ejecucion_id, script, simulacion, timeout_seg }] o lista vacía
POST	/rest/v1/rpc/agente_salida	Guarda un grupo de líneas de salida sin duplicarlas.	Cuerpo: { p_token, p_ejecucion, p_lineas: [{ s, f, t }] }	204 No Content
POST	/rest/v1/rpc/agente_finalizar	Cierra la ejecución con su código y su estado.	Cuerpo: { p_token, p_ejecucion, p_codigo, p_estado }	204 No Content
POST	/rest/v1/ejecuciones_comandos	El panel registra una ejecución nueva; la base solo la acepta si el comando está en el catálogo.	Encabezado: Authorization Bearer (JWT del técnico). Cuerpo: { dispositivo_id, comando_id, modo_simulacion }	201 Created → ejecución con estado pendiente
GET	/rest/v1/comandos	El panel consulta el catálogo de comandos activos.	Encabezado: Authorization Bearer (JWT). Filtro opcional: sistema	200 OK → [{ id, nombre, sistema, destructivo, timeout_seg }]

Fuente: Elaboración propia.

Respuestas y análisis preliminar de riesgos

Cuando la función de ingesta recibe un reporte, contesta con un código que le dice al agente qué pasó y qué tiene que hacer:

Tabla 5. Respuestas de la función de ingesta.

Código	Qué significa	Qué hace el agente
201	El reporte se guardó.	Lo borra de sus pendientes.
200	El reporte ya se había recibido.	Lo toma como enviado y lo borra.
400	El contenido no es un JSON válido.	No reintenta y anota el error.
401	La clave falta, no es válida o ya se cambió.	Deja de enviar y anota el error para que un técnico vuelva a registrar el equipo.
413	El reporte pesa más de 256 KB.	Recorta las listas y lo vuelve a armar.
422	Falta un dato o tiene un formato incorrecto.	No reintenta y anota el error.
429 o 5xx	La plataforma está saturada o tuvo un error.	Lo vuelve a intentar más tarde.

Fuente: Elaboración propia.

Por último, revisé qué podría salir mal en esta comunicación y cómo prevenirlo. La Tabla 6 resume ese análisis preliminar de riesgos.

Tabla 6. Análisis preliminar de riesgos de la API.

Riesgo	Qué podría pasar	Medida preventiva
Se filtra la clave de un equipo	Alguien manda reportes falsos o pide comandos haciéndose pasar por el equipo.	Clave por equipo, guardada solo como huella; cambio de clave al volver a registrar; solo actúa sobre su propio equipo.
Se manipula el panel para ejecutar otra cosa	Ejecutar un comando que no está en el catálogo.	La regla de inserción solo acepta comandos activos del catálogo; el agente nunca recibe texto libre.
Reportes repetidos o reenviados	Datos duplicados y alertas falsas.	Identificador único por reporte y número de orden por línea de salida.
Saturación por muchos equipos	La plataforma responde lento o con errores.	Espera creciente con tiempo al azar entre reintentos y límite de tamaño del reporte.
Reportes mal formados	Datos incorrectos en la base.	Revisión completa del formato antes de guardar (respuesta 422).
Fallas de red en la clínica	Se pierden reportes.	Cola local de 24 horas y envío ordenado al volver la red.

Fuente: Elaboración propia.

3. RESULTADOS DE LA ACTIVIDAD

Al terminar esta actividad quedó definido el contrato completo entre el agente y la plataforma:

Tabla 7. Resumen del contrato de ingesta.

Elemento	Resultado
Formato del reporte	JSON versión 1.0, el mismo para Windows y Mac.
Identificación del equipo	Una clave por equipo, guardada solo como huella y con posibilidad de cambiarla.
Duplicados	Evitados con un identificador por reporte y un número de orden por línea (este último ya implementado).
Reintentos	Espera creciente hasta 60 segundos y cola local de 24 horas.
Funciones de la API	Seis funciones documentadas; las tres del agente para comandos ya funcionan en la demostración.
Diagramas	Dos diagramas de secuencia: envío de reporte y ejecución de un comando.

Elemento	Resultado
Riesgos	Seis riesgos identificados con su medida preventiva.

Fuente: Elaboración propia.

4. PENDIENTES Y SIGUIENTES PASOS

- Confirmar con la asesora la frecuencia del reporte (5 minutos) y la de consulta de comandos.
- Validar el tamaño máximo del reporte y el de la cola de pendientes.
- Saber si los clientes usan proxy para salir a internet.
- Decidir si, además de la clave por equipo, se va a firmar cada mensaje.

Con este contrato se completan las piezas técnicas del entregable E1. En la actividad A1.6 las junto en el documento de arquitectura que se presentará a la asesora para su aprobación.

REFERENCIAS

Bitlab-IT. (2026). Protocolo de estadía profesional (versión 1.0).

Bray, T. (Ed.). (2017). The JavaScript Object Notation (JSON) data interchange format (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Kleppmann, M. (2017). Designing data-intensive applications. O'Reilly Media.

Supabase. (2024). Supabase documentation: Edge Functions and database functions. https://supabase.com/docs